Part 1: Shared Gateway Problem

The Problem: 25 customers generate >20 requests/sec, but the external API allows only 3 requests/sec.

Step 1.1: Goal:

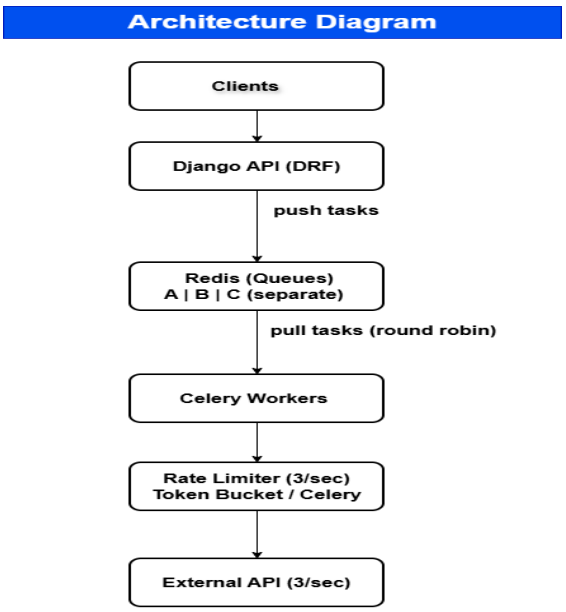
- Limit requests to ≤ 3/sec
- Prevent request loss
- Ensure fairness across customers
- Handle failures reliably

Step 1.2: Architecture:

Components:

- Django API → receives requests
- Redis → stores queues
- Celery → processes requests
- Rate Limiter → controls API usage

Flow:



Step 1.3: Rate Limiting

- Use Token Bucket / Celery rate_limit
- Only 3 requests/sec allowed
- Extra requests → queued

Step 1.4: Fairness

- Use separate queues per customer
- Apply Round Robin scheduling

  A → B → C → A → B → C

Step 1.5: Failure Handling:

- Use Exponential Backoff
  - 1s → 2s → 4s → 8s
- Max retries: 5
- Store failed tasks in Dead Letter Queue

Part 2 — Mobile Architecture

The Problem: Build a **mobile version of SymFlow** that is fast, intuitive, and optimized for real-world logistics usage (drivers, field operations).

Step 1.1: Goal:

Design a fast, simple, low-friction mobile app for logistics users (drivers & customers).

Step 1.2: Tech Stack (Chosen: React Native):

- Framework: React Native
- Backend: Django REST APIs
- State Management: Context API / Redux
- Storage: AsyncStorage (local), AWS S3 (images)
- Authentication: JWT
- Notifications: Firebase Cloud Messaging

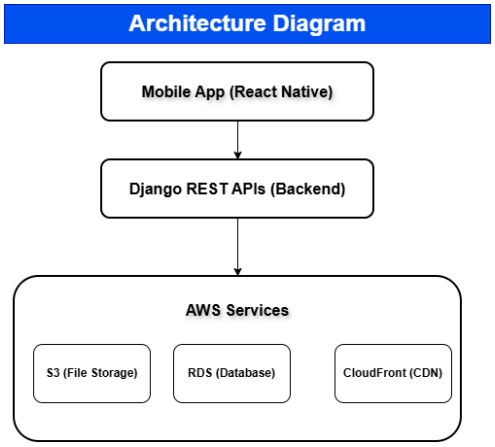
Step 1.3: Why React Native?

- Single codebase for Android & iOS
- Faster development for startup
- Reuse existing React knowledge
- Strong community & ecosystem

Step 1.4: Interaction Model (User Experience)

- **Large Buttons:** Easy for drivers in real conditions
- **Minimal Steps:** Reduce clicks for key actions (upload, confirm delivery)
- **Swipe Actions:** Faster navigation (mark delivered, next task)
- **Voice Input (Optional):** Hands-free operation using AI
- **Offline Mode:** Store data locally → sync when internet is available

Step 1.5: App Architecture:



Part 3 — CI/CD Explanation + GitHub Actions YAML

The Problem: Build a **mobile version of SymFlow** that is fast, intuitive, and optimized for real-world logistics usage (drivers, field operations).

Step 1.1: CI/CD Goal:

Automate deployment for **staging** and **production** to ensure:

- Faster releases
- No manual errors
- Consistent deployments

Step 1.2: CI/CD Pipeline Steps:

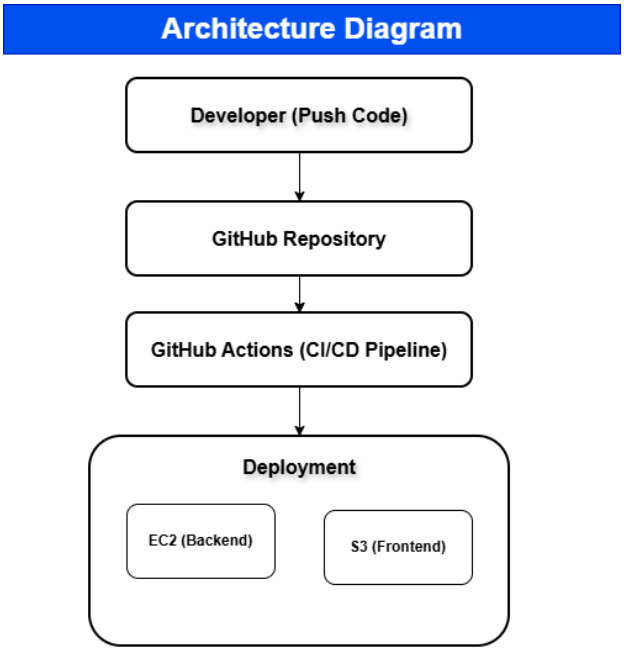
1. **Trigger**
   - On push to staging / main
2. **Checkout Code**
   - Pull latest code from GitHub
3. **Install Dependencies**
   - Backend → pip install
   - Frontend → npm install
4. **Build Frontend**
   - Generate static files
5. **Deploy Frontend**
   - Upload to **AWS S3**
6. **Deploy Backend**
   - SSH into EC2
   - Pull latest code
   - Run migrations
   - Restart services (Gunicorn, Celery)

Step 1.3: GitHub Actions YAML?:

```yaml
name: Deploy to Staging
on:
  push:
    branches:
      - staging

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v3
      - name: Install Backend Dependencies
        run: pip install -r requirements.txt
      - name: Install Frontend Dependencies
        run: npm install

      - name: Build Frontend
        run: npm run build

      - name: Upload to S3
```

```
      run: aws s3 sync build/ s3://your-bucket-name

    - name: Deploy Backend to EC2
      uses: appleboy/ssh-action@v0.1.0
      with:
        host: ${{ secrets.EC2_HOST }}
        username: ubuntu
        key: ${{ secrets.EC2_KEY }}
        script: |
          cd app
          git pull
          pip install -r requirements.txt
          python manage.py migrate
          sudo systemctl restart gunicorn
sudo systemctl restart celery
```

Step 1.4: Current Flow:



Part 4 — Step-by-Step Debugging Flow

The Problem: POD (Proof of Delivery) image upload is failing.

Step 1: Tech Check Django API (Start Point):

- Verify request is reaching server
- Check if image upload to S3 is successful

Step 2: Check S3 Upload:

- Verify task is triggered after upload
- Check if tasks are failing or stuck

Step 3: Check Celery Workers:

- Verify task is triggered after upload
- Check if tasks are failing or stuck

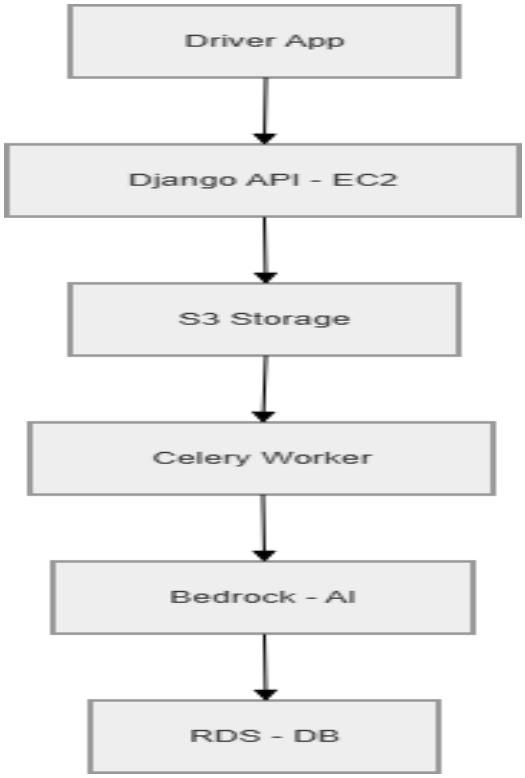
Step 4: Check AI Service (Bedrock):

- Look for timeout or API errors
- Check logs in CloudWatch

Step 5: Check Database (RDS):

- Verify result is saved
- Check DB connection issues

Debugging Strategy:

- Follow request flow step-by-step
- Identify exact failure point
- Fix root cause (not assumptions)

Flow: